

THE ONE-LINER

A consistency model is the contract about what a read can observe after writes — and because replication is asynchronous, reads can be stale or out-of-order unless the system coordinates. The models run from linearizable (one copy, every read fresh) through causal and the session guarantees to eventual (converge, may be stale), and stronger costs latency and availability.

THE SPINE — WHAT YOU DRAW**1. Why does having more than one copy create a consistency problem?**

Because replication is asynchronous: a write commits on the primary and propagates to the other replicas afterward, so for a window they disagree and a read to a lagging replica returns a stale value (and concurrent writes to different replicas can conflict). You replicate for availability, read scaling, and locality — but each replica is a copy that can be behind. Eliminating the staleness needs synchronous coordination on every write, which sacrifices the latency and availability you replicated to gain — so more-than-one-copy forces the trade, and the consistency model names which side you took. Draw the second copy *first*: every anomaly on this board is “which copy answered, and how far behind was it.”

2. Draw the spectrum — and label each model by the anomaly it rules out, not by its name

Four boxes, strongest to weakest, each labelled with what it **forbids**. **Linearizable** — forbids the stale read: a completed write is visible to every read that starts after it (one copy, real-time order). **Causal** — forbids effect-without-cause: no reply visible without its message; unrelated concurrent writes may still be seen in either order. **Session guarantees** — forbid the two things one *user* notices: their own write vanishing (read-your-writes) and time running backwards on refresh (monotonic reads). **Eventual** — forbids nothing except permanent divergence: replicas converge, and that is the entire promise. Labelling by anomaly rather than by name is the whole trick, because it tells you which model to buy: match the cheapest model that forbids the bug you actually have.

3. What does linearizable actually require — and why isn't “one leader” enough?

Write it as: *every operation appears to take effect atomically at some instant between its start and its finish, consistent with real time*. A single leader gives you a **total order** — but not linearizability, because a leader that has been **deposed during a partition and has not learned it yet** will happily serve reads from its own memory while the new leader is already accepting writes it has never seen. So draw the read path as: client to leader, and then a **quorum confirmation** before the leader is allowed to answer (etcd calls this ReadIndex). That extra round trip is the price of the guarantee. The tell of a shallow answer is “we have one primary, so we're strongly consistent”; the tell of a real one is “the node answering must *prove* it is still the leader.”

4. The session guarantees — draw where each one sits in the read path

Draw one user, one write to the primary, and a fan of replicas at different lag. **Read-your-writes**: their next read must not land on a replica behind their own write — so either pin them to the primary briefly, or carry the write's **LSN** and route to a replica that has replayed past it. **Monotonic reads**: their *successive* reads must not go backwards — so pin the session to one replica, or track